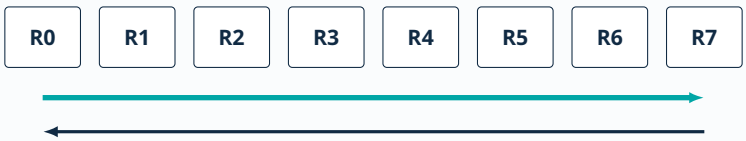
HIGH-PERFORMANCE

16-BIT ARCHITECTURE

LIM M2

Архітектурний довідник процесора

Високодетермінована 16-розрядна мікропрограмна архітектура з чистим мультиплексорним трактом даних та розширеною 24-бітною адресацією.



POINT-TO-POINT DATA ROUTING · ZERO BUS CONTENTION

OFFICIAL REFERENCE SPECIFICATION

Редакція 0.2 — Жовтень 2026 р.

Специфікація процесора LIM M2. Всі права захищено.

Contents

1 Статус документа та архітектурна концепція 3
1.1 Умовні позначення . . . 3
1.2 Єдина філософія формату інструкцій . . . 3

2 Архітектурний огляд процесора LIM M2 3
2.1 Концептуальна схема тракту даних . . . 5

3 Програмно доступні регістри та стан процесора 6
3.1 Карта регістра прапорців та керування (FREG) . . . 6

4 Формат машинних команд 7
4.1 Універсальний бітовий шаблон машинної команди . . . 7
4.2 Клас I: Однобайтові інструкції (1 байт, 0 додаткових октет) . . . 7
4.3 Клас II: Трьохоперандні арифметичні інструкції (2 байти) . . . 7
4.4 Клас III: Двохоперандні логічні інструкції (2 байти) . . . 8
4.5 Клас IV: Міжрегістрове переміщення LRR (2 байти) . . . 8
4.6 Клас V: Апаратний компаратор CMP (2 байти) . . . 8
4.7 Клас VI: Апаратний циклічний зсув на D-тригерах (BSL, BSR) . . . 8
4.8 Клас VII: Операції зі стеком (PUSH, POP) . . . 9
4.9 Клас VIII: Маніпуляція спеціальними регістрами (MSP, Опкод 26) . . . 9
4.9.1 Формат 8.1: Завантаження у флаговий регістр FREG (MOD = 0, REG = 00) . 9
4.9.2 Формат 8.2: Побітова логіка FREG (MOD = 1, REG = 00) . . . 9
4.9.3 Формат 8.3: Завантаження покажчиків PTREG та SREG (MOD = 0, REG = 01 / 10) . . . 10
4.9.4 Формат 8.4: Арифметика зміщення PTREG та SREG (MOD = 1, REG = 01 / 10) 10
4.9.5 Формат 8.5: Завантаження 32-бітних векторних регістрів (MOD = 0, REG = 11) 10
4.10 Клас IX: Інструкції розгалуження та переходів (2–3 байти) . . . 11
4.11 Клас X: Інструкції обміну з пам’яттю LMR та LRM (2–4 байти) . . . 11

5 Зведена таблиця команд ядра LIM M2 11

6 Детальна специфікація системи команд 12
NOP — Апаратна порожня операція та скидання ланцюжка обчислень . . . 13
ADD — Додавання з автоматичним каскадуванням переносу . . . 15
SUB — Віднімання з автоматичним каскадуванням позики . . . 17
MUL — Цілочисельне множення зі збереженням старшої частини в регістрі АЛП . . . 19
DIV — Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП 21
LMR — Завантаження даних із пам’яті та зовнішніх пристроїв у регістри . . . 23
LRM — Запис даних із регістрів у пам’ять та зовнішні пристрої . . . 25

LRR — Міжрегістрове переміщення (копіювання з точки Б в точку А) 27
CMP — Апаратне порівняння операндів 29
JMP — Безумовний перехід 31
JCR — Умовний перехід за прапорцем переносу 33
JNZ — Умовний перехід, якщо не нуль 35
JZ — Умовний перехід за нулем 37
AND — Побітове логічне І 39
OR — Побітове логічне АБО 41
NAND — Побітове логічне І-НІ 43
NOR — Побітове логічне АБО-НІ 45
NOT — Побітова інверсія 47
XOR — Побітове виключне АБО 49
XNOR — Побітова еквівалентентність 51
INC — Інкремент на одиницю 53
DEC — Декремент на одиницю 55
PUSH — Збереження слова в стек 57
POP — Вилучення слова з вершини стека 59
BSL — Апаратний зсув ліворуч (ітеративний регістр) 61
BSR — Апаратний зсув праворуч (ітеративний регістр) 63
MSP — Маніпуляція спецрегістрами (FREG, PTREG, SREG, VEC) 65
MOD — Цілочисельний залишок від ділення (modulo) 67
INTR — Програмне переривання за таблицею векторів (2 байти) 69
CALL — Виклик підпрограми зі збереженням адреси повернення 71
RET — Повернення з підпрограми з відновленням PC зі стека 73
HALT — Апаратний зупин тактування процесора 75

7 Фізичний інтерфейс та цоколівка (50-вивідний корпус) 77
7.1 Опис системних сигналів 77

8 Підсистема переривань та векторна обробка 78
8.1 Єдиний простір 16 векторів переривань 78
8.2 Апаратний протокол квітування (RDI → IA → D0–D7) 78
8.3 Збереження контексту у 24-бітному стеку та вхід в обробник 79
8.4 Повернення з переривання 79

9 Мова асемблера та компілятор LIM M2 (`limasm.py`) 80
9.1 Іменування регістрів та суфікси розміру операндів 80
9.2 Арифметичні, унарні інструкції та робота з пам’яттю (LMR, LRM) 80
9.3 Спеціальні регістри (MSP) та багаторазове розіменування через PTREG 80

10 Висновки 81

1 Статус документа та архітектурна концепція

Цей документ є офіційним технічним описом та архітектурним стандартом 16-розрядного мікропроцесора **LIM M2**. Архітектура спроєктована з фокусом на абсолютну передбачуваність, математичну точність і високу тактову частоту.

Ключовим рішенням у тракті даних LIM M2 стала **повна відмова від внутрішніх шин третього стану (Tri-State / Hi-Z)**. Внутрішні потоки даних побудовані на основі активних мультиплексорних дерев (Point-to-Point MUX Routing), що цілком усуває ризик конфліктів шини (bus contention), наскрізних струмів та паразитних ємнісних затримок.

> **Нумерація системи команд**
> Система команд LIM M2 використовує суцільний 5-бітний діапазон базових опкодів: `NOP`=0, `ADD`=1, ..., `MSP`=26. Опкоди 27–31 зарезервовано для розширень наступного покоління.

1.1 Умовні позначення

Позначення	Архітектурне значення
`dst`	Цільовий регістр-приймач результату операції
`src`, `srcA`, `srcB`	Регістри-джерела вхідних операндів
`imm`	Безпосереднє скалярне значення в тілі команди
`addr`	Ефективна адреса пам'яті (16-бітна базова або 24-бітна розширена)
`cfg`	3-бітне поле конфігураційного модифікатора операції
`ZF`, `CF`, `OF`	Апаратні прапорці нуля, переносу та переповнення регістра `FREG`

1.2 Єдина філософія формату інструкцій

Архітектура процесора LIM M2 суворо дотримується універсальної парадигми: **спочатку завжди вказується цільовий приймач (куди), а потім вхідні операнди (звідки)**. Це забезпечує стрункий, логічний та інтуїтивно зрозумілий синтаксис:

- **Арифметичні операції (3 операнди):** `ADD Rd, Rs1, Rs2` означає Rd $\leftarrow$ Rs1 $+$ Rs2 (наприклад, `ADD R0, R1, R2` обчислює $R0 = R1 + R2$); аналогічно для `SUB`, `MUL`, `DIV`.
- **Логічні операції (2 операнди):** `AND Rd, Rs` означає Rd $\leftarrow$ Rd AND Rs (наприклад, `AND R0, R1` обчислює $R0 = R0 \,\&\, R1$); аналогічно для `OR`, `NAND`, `NOR`, `XOR`, `XNOR`.
- **Однобайтові унарні операції (1 операнд):** `INC Rd` (Rd $\leftarrow$ Rd $+ 1$), `DEC Rd` (Rd $\leftarrow$ Rd $- 1$) та `NOT Rd` (Rd $\leftarrow \sim$ Rd). Працюють із єдиним цільовим регістром Rd на місці. Кодуються рівно в 1 байті (`[7:3]` опкод, `[2:0]` регістр Rd), другий байт із шини пам'яті не запитується.
- **Міжрегістрові пересилання:** `LRR Rd, Rs` означає пряме копіювання з точки Б (джерело Rs) у точку А (приймач Rd): Rd $\leftarrow$ Rs (наприклад, `LRR R0, R1` означає $R0 = R1$).
- **Завантаження з пам'яті (LMR):** перший аргумент Rd визначає цільовий регістр процесора, куди записуються зчитані дані.

2 Архітектурний огляд процесора LIM M2

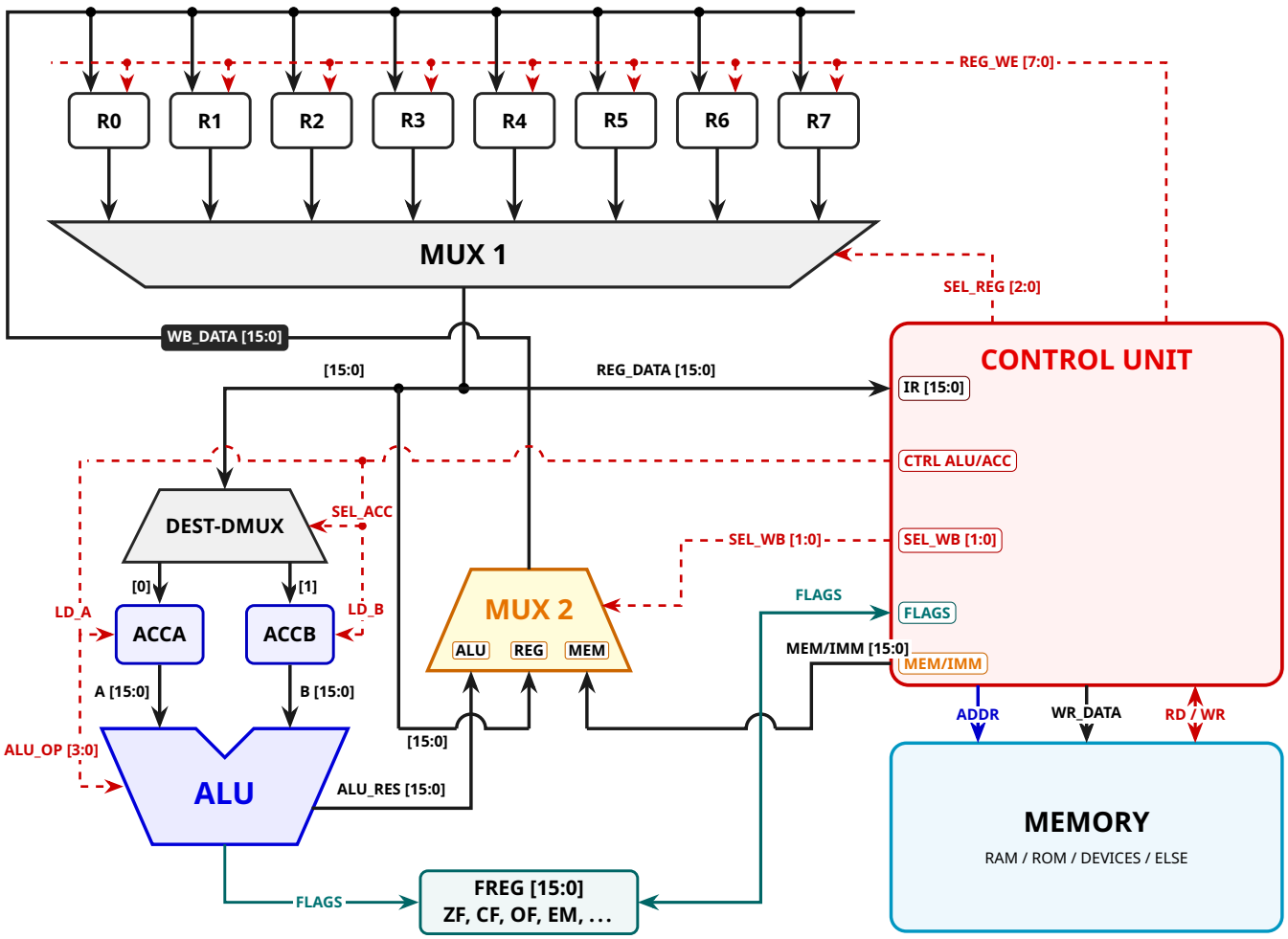
LIM M2 — високопродуктивний 16-розрядний процесор із мікропрограмним керуванням, орієнтований на цілочисельні обчислення. Основні архітектурні переваги:

- **Чиста мультиплексорна маршрутизація:** Тракт даних використовує виключно активні

драйвери через каскад селекторів **MUX 1**, **DEST-DMUX** та **MUX 2**. Це гарантує ідеальну сумісність із кремнієвими технологіями (ASIC) та сучасними ПЛІС (FPGA).

- **Двоакумуляторний виконавчий вузол:** Регістри ACCA та ACCB надійно фіксують операнди перед АЛП, ізолюючи перехідні процеси від регістрового файлу.
- **Апаратне каскадування довгої арифметики (ALU Auto-Chaining):** Внутрішній тракт АЛП містить залишковий акумулятор (ALU_RESIDUAL). У разі переносу ($+1$ при сумі $> 2^{16} - 1$), позики (-1 при різниці < 0) або надлишкових старших розрядів множення/ділення, залишок фіксується апаратно та автоматично додається чи віднімається під час наступної математичної операції. Це забезпечує наскрізну багатобайтову арифметику (32, 48, 64+ бітів) без окремих команд ADC/SBB. Скидання ланцюга виконується командою NOP.
- **Прямий канал пересилання (Reg-to-Reg Bypass):** Обхідний шлях із MUX 1 безпосередньо в MUX 2 дозволяє копіювати дані між регістрами за мінімальний час, не блокуючи АЛП.
- **Нативна 32-бітна адресація пам'яті (до 4 ГБ):** Нативна 32-розрядна магістраль адреси (A0–A31) та спеціалізовані 32-бітні регістри-вказівники PTREG (вказівник даних з автоінкрементом) і SREG (вказівник стека), а також 32 системні векторні регістри (`$ir0..$ir15`, `$pv0..$pv15`) забезпечують пряме лінійне звернення до всієї фізичної пам'яті об'ємом 4 Гігабайти без мультиплексування шин, перемикання сторінок чи банків.
- **Високий потенціал тактової частоти:** Завдяки відсутності ємнісного навантаження розподілених шин ядро розраховане на базову частоту **200 МГц**, а в оптимізованому кремнії досягає **700 МГц і вище**.

2.1 Концептуальна схема тракту даних



3 Програмно доступні регістри та стан процесора

Архітектурний стан процесора LIM M2 утворює чітку ортогональну систему регістрів:

Регістр	Розрядність	Призначення та ключові особливості
R0–R7	16 бітів	Регістри загального призначення (GPR). Селектор MUX 1 забезпечує швидке зчитування; запис відбувається за індивідуальними стробами з шини WB_DATA.
SREG	24 біти	Системний регістр стека (Stack Register). Забезпечує апаратну підтримку викликів процедур і кадрів стека з прямою 24-бітною адресацією стекового простору (до 16 Мбайт).
PTREG	24 біти	Високоточний регістр-покажчик (Pointer Register). Забезпечує суцільну адресацію пам'яті до 16 Мбайт з апаратним автоінкрементом за прапорцем AIP.
FREG	16 бітів	Регістр стану та керування (Processor Status & Control Register). Об'єднує арифметичні прапорці результату та біти конфігурації ядра.

3.1 Карта регістра прапорців та керування (FREG)

Регістр FREG координує функціонування обчислювального ядра:

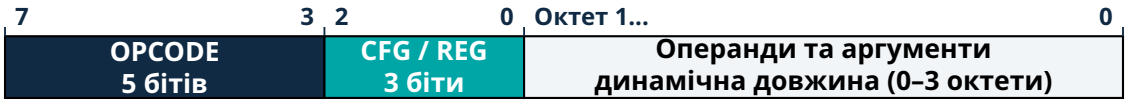
Біт	Мнемоніка	Функціональний опис
0	ZF	Zero Flag: істинний при нульовому результаті операції.
1	CF	Carry Flag: апаратний перенос або позика у знаковому/беззнаковому режимах.
2	OF	Overflow Flag: індикатор знакового переповнення додаткового коду.
3	EM	Error Math: апаратний прапорець математичних помилок (ділення на нуль).
4–5	CMP	Compare Result: результат апаратного компаратора (00: скидання/не порівнювалось/помилка, 01: менше, 10: дорівнює, 11: більше).
6–7	CLK	Clock Mode: конфігурація тактового генератора та дільників частоти.
8	PIE	Program Interrupt Enable: глобальний дозвіл програмних переривань.
9	MIE	Master Interrupt Enable: дозвіл зовнішніх апаратних переривань.
10	DWR	Disable Wait for Ready: форсований обмін без очікування сигналу готовності шини.
11	PRV	Privilege Mode: 0 = User Mode, 1 = Supervisor Mode (апаратний старт в 1; захист через 8 Зон Привілеїв $pv0..$pv15).
12	AIP	Auto-Increment PTREG: дозвіл апаратного автоінкременту покажчика PTREG.
13	DBIT	Displaced Bit: буфер витісненого біта зсувного вузла.
14	SMOD	Shift Mode: керування циклічним режимом зсуву (0: заповнення нулями, 1: прохід через DBIT).
15	INTR	In Interrupt: прапорець перебування у перериванні (при 1 нові переривання апаратно заблоковані).

4 Формат машинних команд

Інструкції мікропроцесора LIM M2 мають ортогональний, модульний формат із детермінованим апаратним декодуванням. Базовий опкод завжди займає старші 5 бітів першого октету, що забезпечує миттєву диспетчеризацію в мікропрограмному керуючому ПЗП ядра.

4.1 Універсальний бітовий шаблон машинної команди

Кожна машинна інструкція починається з базового заголовка фіксованого розміру, за яким слідують динамічні операнди та аргументи:



Молодші 3 біти першого октету (`[2:0]`) мають подвійне детерміноване призначення:

- В операціях з регістрами (арифметика, унарні команди, логіка, зсуви, пересилання, стек) біти $[2:0]$ безпосередньо кодують **перший регістровий аргумент** (Rd або Rs1), що дозволяє адресувати повний набір із 8 регістрів R0–R7 без додаткових накладних витрат.
- У командах, які потребують вибору апаратного режиму чи селектора пам'яті/спецрегістрів (LMR, LRM, MSP, JMP, JCR, JNZ, JZ), біти $[2:0]$ виступають конфігуратором режиму (CFG / MOD / SRC / DST).
- У безаргументних керуючих інструкціях (NOP, HALT, CLI, STI, RET) ці біти містять `000`.

4.2 Клас I: Однобайтові інструкції (1 байт, 0 додаткових октет)

Інструкції керування (NOP, HALT, CLI, STI, RET) та унарні регістрові операції (NOT, INC, DEC) зчитуються за рівно один такт вибірки. Другий байт із шини пам'яті не запитується:

- Для безаргументних команд молодші 3 біти містять `000`.
- Для унарних команд (`INC Rd`, `DEC Rd`, `NOT Rd`) молодші 3 біти (`[2:0]`) містять індекс цільового регістра Rd (R0–R7, єдиний аргумент).



Октет 0: Фіксована довжина 1 байт

4.3 Клас II: Трьохоперандні арифметичні інструкції (2 байти)

Команди обчислень (ADD, SUB, MUL, DIV, MOD) суворо реалізують архітектурну парадигму **«спочатку приймач, потім доданки»**:

$$\texttt{ADD Rd, Rs1, Rs2} \implies \texttt{Rd} \leftarrow \texttt{Rs1} + \texttt{Rs2} + \texttt{ALU_RESIDUAL}$$

$$\texttt{MOD Rd, Rs1, Rs2} \implies \texttt{Rd} \leftarrow \texttt{Rs1} \pmod{\texttt{Rs2}}$$

Перший аргумент команди (цільовий регістр призначення Rd) розміщується безпосередньо в молодших 3 бітах першого октету, що дозволяє адресувати повні 8 регістрів R0–R7 для всіх трьох операндів:

- **Октет 0:** біти $[7:3]$ — код операції (5 бітів); біти $[2:0]$ — цільовий регістр призначення Rd (3 біти, R0–R7, перший аргумент).
- **Октет 1:** біти $[7:5]$ — перший операнд-джерело Rs1 (3 біти, R0–R7); біти $[4:2]$ — другий операнд-джерело Rs2 (3 біти, R0–R7); біти $[1:0]$ — резерв (`00`).



4.4 Клас III: Двохоперандні логічні інструкції (2 байти)

Логічні операції (AND, OR, NAND, NOR, XOR, XNOR) мають рівно два операнди:

$$\texttt{AND Rd, Rs} \quad \Longrightarrow \quad \text{Rd} \leftarrow \text{Rd AND Rs}$$

- **Октет 0:** біти $[7:3]$ — код операції; біти $[2:0]$ — цільовий регістр-приймач Rd (R0–R7, перший аргумент).
- **Октет 1:** біти $[7:5]$ — другий регістр-операнд Rs (R0–R7); біти $[4:0]$ — зарезервовані (00000_2).



4.5 Клас IV: Міжрегістрове переміщення LRR (2 байти)

Інструкція LRR Rd, Rs копіює дані з джерела (точка Б, Rs) у приймач (точка А, Rd):

$$\texttt{LRR Rd, Rs} \quad \Longrightarrow \quad \text{Rd} \leftarrow \text{Rs}$$

Переміщення здійснюється через прямий апаратний обхідний тракт **MUX 1** $\rightarrow$ **MUX 2** повз АЛП за один мікротакт, без зміни регістра прапорців FREG. Формат октетів: Октет 0 містить опкод `00111` та цільовий регістр Rd (`[2:0]`), Октет 1 містить регістр-джерело Rs (`[7:5]`) та резерв (`[4:0]` = `00000`).

4.6 Клас V: Апаратний компаратор CMP (2 байти)

Інструкція CMP Rs1, Rs2 порівнює значення регістрів без руйнування вмісту:

- **Октет 0:** код операції `01000`, біти $[2:0]$ задають перший операнд-джерело Rs1 (перший аргумент).
- **Октет 1:** біти $[7:5]$ задають другий операнд-джерело Rs2; біти $[4:0]$ містять `00000`.

Результат порівняння фіксується у спеціалізованому 2-бітному апаратному полі регістра стану FREG.CMP (біти $[5:4]$):

Значення FREG.CMP	Умова	Архітектурний зміст
`01` (01_2)	Rs1 $<$ Rs2	Перший операнд суворо менший за другий
`10` (10_2)	Rs1 $=$ Rs2	Значення операндів рівні
`11` (11_2)	Rs1 $>$ Rs2	Перший операнд суворо більший за другий
`00` (00_2)	Вихідний / Скидання	Нічого не порівнювалося / помилка компаратора

4.7 Клас VI: Апаратний циклічний зсув на D-тригерах (BSL, BSR)

Команди зсуву BSL Rd, shift та BSR Rd, shift реалізовані у кремнії як **ітеративний 16-розрядний регістр зсуву на D-тригерах** (≈ 300 транзисторів замість $\approx 30\,000$ у матричного барельного вузла):

- Зсув виконується апаратно такт-за-тактом (`for` у залізі): зсув на 16 бітів потребує рівно на 15 тактів більше, ніж зсув на 1 біт.
- Октет 0: старші 5 бітів задають опкод (BSL: `11000`, BSR: `11001`), а молодші 3 біти (`[2:0]`) прямо вказують зсувний регістр Rd.
- Октет 1: весь байт (`[7:0]`) вказує кількість тактів зсуву shift (0–16).
- Поведінка бітів визначається регістром FREG: при SMOD $= 1$ увімкнено апаратну ротацію через витіснений біт; при SMOD $= 0$ порожні розряди заповнюються бітом із `FREG.DBIT`.



Октет 0: Опкод + Rd — Октет 1: Кількість тактів зсуву (1 такт на біт)

4.8 Клас VII: Операції зі стеком (PUSH, POP)

Команди стека зберігають регістр у пам'ять за 32-бітним вказівником SREG:

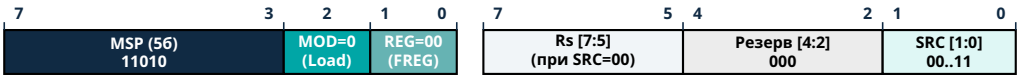
- **Октет 0:** код операції (PUSH: `10110`, POP: `10111`), біти $[2:0]$ прямо задають регістр операнда (Rs при збереженні, Rd при вилученні, перший аргумент).
- **Октет 1:** біти $[7:0]$ містять 00000000_2 (резерв).

4.9 Клас VIII: Маніпуляція спеціальними регістрами (MSP, Опкод 26)

Інструкція MSP (Опкод `11010`, 26_{10}) забезпечує універсальний модульний доступ до всіх системних, адресних та захисних ресурсів ядра: FREG, PTREG, SREG, а також 32 системних векторів переривань та меж Зон Привілеїв (`$ir0..$ir15`, `$pv0..$pv15`). Довжина команди становить від 2 до 6 байтів залежно від обраного режиму адресації.

4.9.1 Формат 8.1: Завантаження у флаговий регістр FREG (MOD = 0, REG = 00)

Команда `MSP.ld $freg, <src>` виконує пряму ініціалізацію прапорців.

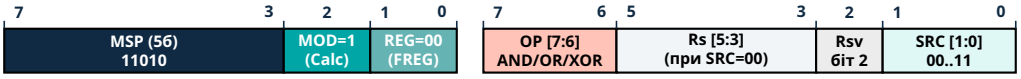


Октет 0: Опкод 26 + MOD=0 + FREG (0xD0) — Октет 1: Селектор джерела (довжина 2–4 байти)

- SRC $= 00_2$ (2 байти): завантаження з регістра РЗП Rs (`MSP.ld $freg, $rs`).
- SRC $= 01_2$ (4 байти): безпосередня константа imm16 в октетах 2–3 (`MSP.ld $freg, 0x0000`).
- SRC $= 10_2$ (2 байти): читання з пам'яті за вказівником `[$ptreg]` (`MSP.ld $freg, [$ptreg]`).
- SRC $= 11_2$ (4 байти): читання з пам'яті за прямою 16-бітною адресою `[addr16]` (`MSP.ld $freg, [0x1000]`).

4.9.2 Формат 8.2: Побітова логіка FREG (MOD = 1, REG = 00)

Команди `MSP.and`, `MSP.or`, `MSP.xor`, `MSP.xnor` застосовують побітову маску до прапорців.



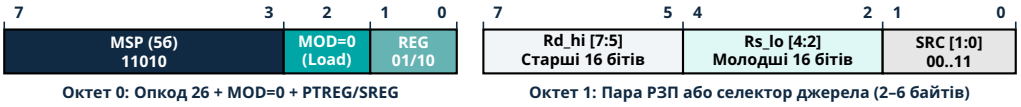
Октет 0: Опкод 26 + MOD=1 + FREG (0xD4) — Октет 1: Код операції OP + Селектор маски

- **Код операції OP**$[7:6]$**:** 00_2 = AND, 01_2 = OR, 10_2 = XOR, 11_2 = XNOR.

- **Апаратний захист режиму супервізора (PRV):** Процесор апаратно стартує під час увімкнення живлення та скидання в режимі супервізора (PRV=1). Початковий код (завантажувач/ядро ОС) налаштовує вектори (`$ir0..$ir15`) та межі Зон Привілеїв (`$pv0..$pv15`), після чого скидає PRV=`0` і передає керування користувацькому коду. У користувацькому режимі (PRV=`0`) увімкнення PRV=1 та модифікація зон апаратно дозволені виключно якщо поточний PC знаходиться суворо всередині однієї з 8 зареєстрованих Зон Привілеїв ($\exists k \in [0..7] : \$pv_{2k} \leq \text{PC} \leq \$pv_{2k+1}$). **Важливе архітектурне правило:** під час виходу за межі Зони Привілеїв режим супервізора PRV=1 автоматично **не скидається** — ОС/програма зобов'язана явно скинути його перед поверненням користувачеві.

4.9.3 Формат 8.3: Завантаження покажчиків PTREG та SREG (MOD = 0, REG = 01 / 10)

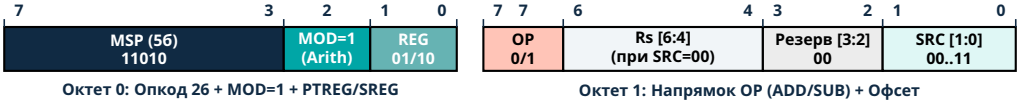
Команди `MSP.ld $ptreg` (REG $= 01_2$, опкод `0xD1`) та `MSP.ld $sreg` (REG $= 10_2$, опкод `0xD2`) виконують пряму ініціалізацію 32-бітних лінійних покажчиків.



- SRC $= 00_2$ (2 байти): збирання адреси з пари РЗП `Rd_hi:Rs_lo` (`MSP.ld $ptreg, [$0:$1]`).
- SRC $= 01_2$ (4–6 байтів): безпосередня адреса `imm32` (6 байтів) або `imm16` (4 байти).
- SRC $= 10_2$ (**2 байти**): **нативне 1-тактове розіменування покажчика** `[$ptreg]` (`PTREG <- MEM[PTREG]`).
- SRC $= 11_2$ (4–6 байтів): завантаження покажчика з пам'яті за адресою `[addr]` (`MSP.ld $ptreg, [0xA000]`).

4.9.4 Формат 8.4: Арифметика зміщення PTREG та SREG (MOD = 1, REG = 01 / 10)

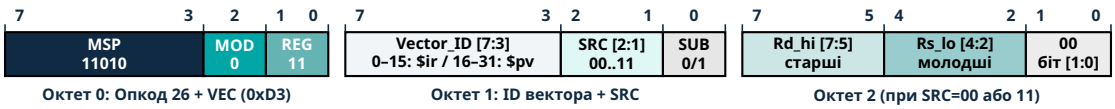
Команди `MSP.add` (OP $= 0$) та `MSP.sub` (OP $= 1$) здійснюють швидке апаратне позиціонування покажчиків буфера та вершини стека.



- OP $= 0$: ADD (+зміщення до покажчика), OP $= 1$: SUB (-зміщення від покажчика).
- Джерело зміщення: регістр РЗП SRC $= 00_2$ (2 байти), безпосередня константа SRC $= 01_2$ (4 байти), читання зміщення за SRC $= 10_2$ (`[$ptreg]`, 2 байти) або SRC $= 11_2$ (`[addr16]`, 4 байти).

4.9.5 Формат 8.5: Завантаження 32-бітних векторних регістрів (MOD = 0, REG = 11)

Команда `MSP.ld` завантажує 32-бітні адреси в один із 32 векторних регістрів: `$ir0..$ir15` (вектори переривань, ID 0–15) та `$pv0..$pv15` (межі Зон Привілеїв, ID 16–31).



- **Варіант 5.1 (SRC $= 00_2$, SUB $= 0$, 3 байти):** завантаження вектора з пари РЗП `Rd_hi:Rs_lo` (`MSP.ld $ir0, [$0:$1]`).

- **Варіант 5.2 (SRC $= 01_2$, SUB $= 0$, 6 байтів):** прямий 32-бітний вектор адреси `imm32` в октетах 2–5 (`MSP.ld $pv0, 0x00008000`).
- **Варіант 5.3 (SRC $= 10_2$, SUB $= 0$, 2 байти):** читання вектора з пам'яті за покажчиком `[$ptreg]` (`MSP.ld $ir1, [$ptreg]`).
- **Варіант 5.4 (SRC $= 11_2$, SUB $= 0$, 3 байти):** читання вектора з пам'яті за 32-бітним покажчиком у парі РЗП `[$rd:$rs]` (`MSP.ld $ir2, [$0:$1]`).
- **Варіант 5.5 (SRC $= 11_2$, SUB $= 1$, 6 байтів):** читання вектора з пам'яті за прямою 32-бітною адресою `[addr32]` (`MSP.ld $ir0, [0x00005000]`).

4.10 Клас IX: Інструкції розгалуження та переходів (2–3 байти)

Команди переходів (`JMP`, `JCR`, `JNZ`, `JZ`) підтримують три взаємовиключні режими адресації, що задаються полем CFG (біти $[2:0]$ октету 0):

- **MOD = 0 (`000`): Безпосередня 16-бітна адреса (довжина 3 байти).** За октетом 0 слідують октети 1 та 2, що формують абсолютну 16-розрядну адресу цільової точки переходу в поточному 64-КБ сегменті.
- **MOD = 1 (`001`): Непрямий регістровий перехід (довжина 2 байти).** Октет 1 вказує цільовий регістр Rs (біти $[7:5]$), значення якого завантажується в PC.
- **MOD = 2 (`010`): Розширений 24-бітний перехід за вказівником PTREG.** Перехід здійснюється за фізичною 24-бітною адресою із системного регістра PTREG.

4.11 Клас X: Інструкції обміну з пам'яттю LMR та LRM (2–4 байти)

Команди завантаження (`LMR`) та збереження (`LRM`) мають найвищий ступінь адаптивності. Поле конфігурації октету 0 (`[2:0]`) кодує:

- Біт $[2]$: розмір операнда SZ ($0 = 8$ бітів / байт, $1 = 16$ бітів / слово).
- Біти $[1:0]$: джерело/приймач SRC / DST:
  - **0: Безпосередні дані (Inline Data):** 2 байти при SZ $= 0$ (байт за командою) або 3 байти при SZ $= 1$ (16-бітне слово за командою).
  - **1: Безпосередній вказівник (Imm Pointer):** 4 байти. Октет 1 задає робочий регістр, а октети 2–3 зберігають 16-бітну абсолютну адресу даних.
  - **2: Системний 24-бітний вказівник PTREG:** 2 байти. Октет 1 задає цільовий регістр. При активному прапорці `FREG.AIP` вказівник PTREG апаратно інкрементується за 0 додаткових тактів.
  - **3: Складена 24-бітна адреса з пари регістрів:** 2 байти. Октет 1 містить біти $[7:5]$ (`Arg1` / старші біти адреси) та біти $[4:2]$ (`Arg2` / молодші 16 бітів адреси). У команді `LMR` регістр `Arg1` після читання перезаписується зчитаними даними.

5 Зведена таблиця команд ядра LIM M2

№	Код	Мнемоніка	Клас	Функціональне призначення
0	`00000`	`NOP`	Керування	Порожня операція та скидання залишкового регістра АЛП
1	`00001`	`ADD`	Арифметика	Додавання з автокаскадуванням переносу через залишок АЛП
2	`00010`	`SUB`	Арифметика	Віднімання з автокаскадуванням позики через залишок АЛП
3	`00011`	`MUL`	Арифметика	Множення з фіксацією старших розрядів у залишку АЛП
4	`00100`	`DIV`	Арифметика	Ділення з фіксацією залишку у внутрішньому регістрі АЛП

№	Код	Мнемоніка	Клас	Функціональне призначення
5	00101	LMR	Пам’ять	Завантаження з пам’яті/вводу в регістр (конфіг джерела 0–3, розмір 8/16 біт)
6	00110	LRM	Пам’ять	Запис даних з регістра в пам’ять/вивід (конфіг призначення 0–3, розмір 8/16 біт)
7	00111	LRR	Пересилання	Пряме міжрегістрове копіювання через обхідний MUX
8	01000	CMP	Порівняння	Апаратне порівняння без спотворення операндів
9	01001	JMP	Перехід	Безумовний перехід (прямий, непрямий, 24-бітний)
10	01010	JCR	Перехід	Умовний перехід за прапорцем переносу CF
11	01011	JNZ	Перехід	Умовний перехід, якщо результат не нуль (ZF = 0)
12	01100	JZ	Перехід	Умовний перехід за нульовим результатом (ZF = 1)
13	01101	AND	Логіка	Побітове логічне множення (І)
14	01110	OR	Логіка	Побітове логічне додавання (АБО)
15	01111	NAND	Логіка	Побітове логічне І-НІ
16	10000	NOR	Логіка	Побітове логічне АБО-НІ
17	10001	NOT	Логіка	Побітова інверсія операнда (доповнення до 1)
18	10010	XOR	Логіка	Побітове виключне АБО
19	10011	XNOR	Логіка	Побітова еквівалентність
20	10100	INC	Арифметика	Апаратний інкремент значення на 1
21	10101	DEC	Арифметика	Апаратний декремент значення на 1
22	10110	PUSH	Стек	Розміщення слова у стековому просторі
23	10111	POP	Стек	Вилучення вершини стека в цільовий регістр
24	11000	BSL	Зсуви	Барельний зсув ліворуч із налаштовуваним бітом
25	11001	BSR	Зсуви	Барельний зсув праворуч із фіксацією в DBIT
26	11010	MSP	Система	Маніпуляція службовими регістрами (FREG, PTREG, SREG)
27	11011	MOD	Арифметика	Обчислення залишку від цілочисельного ділення (modulo)
28	11100	INTR / INT	Система	Програмне переривання (2 байти, молодші 4 біти октету 1 — вектор 0..15)
29	11101	CALL	Переходи	Виклик підпрограми зі збереженням 32-бітного PC у стек SREG
30	11110	RET	Переходи	Повернення з підпрограми зі зняттям 32-бітного PC зі стека SREG
31	11111	HALT	Керування	Апаратний зупин процесора (HLT)

6 Детальна специфікація системи команд

Нижче подано повний архітектурний опис усіх 32 опкодів ядра LIM M2.

NOP

Апаратна порожня операція та скидання ланцюжка обчислень

`00000`
Керування

Мнемоніка	NOP
Opcode	`00000` (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Керування
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратна порожня операція та скидання ланцюжка обчислень.

Архітектурна семантика

```
Сигнатура:  NOP
Операція:  PC <- PC + 1, ALU_RESIDUAL <- 0, FREG.CF <- 0, FREG.OF <- 0
Приклад:  NOP ; Скидання ланцюжка переносу АЛП
```

Конфігурація та операнди



Октет 0 (1 байт, 0 дод. байт)

Команда фіксованої довжини (1 байт). Операнди відсутні; поле cfg містить `000` (зарезервовано для калібрування затримок). Другий байт із шини пам’яті не запитується.

Вплив на регістр прапорців (FREG)

Скидає прапорці переносу (CF), переповнення (OF) та позики; інші прапорці зберігаються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Виконується для тактової синхронізації конвеєра та апаратного скидання (Flush) внутрішнього залишкового регістра АЛП, розриваючи ланцюг автопереносу перед новими незалежними обчисленнями.

Поля з позначкою TBD визначають розширені параметри специфікації.

ADD

Додавання з автоматичним каскадуванням переносу

00001
Арифметика

Мнемоніка	ADD
Opcode	00001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

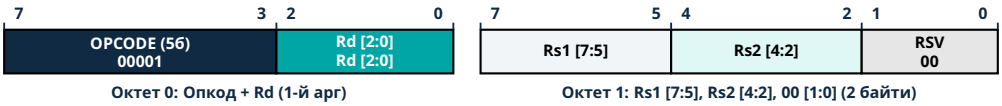
Призначення інструкції

Додавання з автоматичним каскадуванням переносу.

Архітектурна семантика

```
Сигнатура:  ADD arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 + arg2
Операція:  sum <- arg1 + arg2 + ALU_RESIDUAL
arg0 <- sum[15:0], ALU_RESIDUAL <- (sum > 0xFFFF ? 1 :  0)
Приклад:  ADD R0, R1, R2 ; R0 = R1 + R2 (регістри R0-R7)
```

Конфігурація та операнди



Октет 0: код операції 00001, біти $[2:0]$ задають цільовий регістр призначення Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають перший доданок Rs1 (R0–R7); біти $[4:2]$ задають другий доданок Rs2 (R0–R7); біти $[1:0]$ — резерв (00). Довжина команди: рівно 2 байти.

Вплив на регістр прапорців (FREG)

Встановлює прапорець переносу CF та переповнення OF при перевищенні $2^{16}-1$; оновлює ZF за значенням dst.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.

2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Забезпечує наскрізну багатобайтову арифметику (32, 48, 64+ бітів). Якщо результат суми перевищує 16 бітів ($> 2^{16} - 1$), перенос (+1) фіксується у внутрішньому регістрі АЛП та автоматично додається під час наступної операції ADD без потреби в окремих інструкціях ADC. Молодші 16 бітів детерміновано зберігаються в dst. Ланцюг скидається командою NOP або перезаписується новою математичною операцією.

Поля з позначкою TBD визначають розширені параметри специфікації.

SUB

Віднімання з автоматичним каскадуванням позики

00010
Арифметика

Мнемоніка	SUB
Opcode	00010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Віднімання з автоматичним каскадуванням позики.

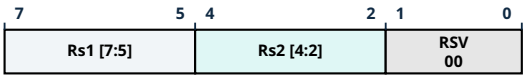
Архітектурна семантика

```
Сигнатура:  SUB arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 - arg2
Операція:  diff <- arg1 - arg2 - ALU_RESIDUAL
arg0 <- diff[15:0], ALU_RESIDUAL <- (diff < 0 ?  1 :  0)
Приклад:  SUB R0, R1, R2 ; R0 = R1 - R2 (регістри R0–R7)
```

Конфігурація та операнди



Октет 0: Опкод + Rd (1-й арг)



Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байти)

Октет 0: код операції 00010, біти $[2:0]$ задають цільовий регістр призначення Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають зменшуване Rs1 (R0–R7); біти $[4:2]$ задають від'ємник Rs2 (R0–R7); біти $[1:0]$ — резерв (00). Довжина команди: рівно 2 байти.

Вплив на регістр прапорців (FREG)

При різниці < 0 встановлює прапорець позики (Borrow / CF) та OF; оновлює ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).

3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Реалізує неперервне багатослівне віднімання. При виникненні від'ємної різниці (< 0) позика (-1) зберігається у внутрішньому регістрі АЛП, і в наступній операції SUB АЛП автоматично віднімає 1 зі старшого слова. Результат молодшого слова детерміновано фіксується в dst. Для початку нової незалежної різниці ланцюг переривається інструкцією NOP.

Поля з позначкою TBD визначають розширені параметри специфікації.

MUL

Цілочисельне множення зі збереженням старшої частини в регістрі АЛП

00011
Арифметика

Мнемоніка	MUL
Opcode	00011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

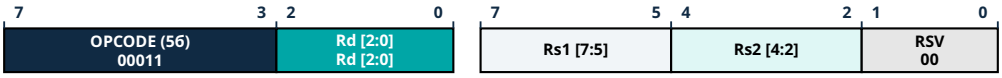
Призначення інструкції

Цілочисельне множення зі збереженням старшої частини в регістрі АЛП.

Архітектурна семантика

```
Сигнатура:  MUL arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 * arg2
Операція:  prod <- arg1 * arg2
arg0 <- prod[15:0], ALU_RESIDUAL <- prod[31:16]
Приклад:  MUL R0, R1, R2 ; R0 = R1 * R2 (старша частина в ALU_RESIDUAL)
```

Конфігурація та операнди



Октет 0: Опкод + Rd (1-й арг) Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байти)

Октет 0: код операції 00011, біти $[2:0]$ задають цільовий регістр призначення Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають перший множник Rs1 (R0–R7); біти $[4:2]$ задають другий множник Rs2 (R0–R7); біти $[1:0]$ — резерв (00). Старші 16 бітів добутку автоматично фіксуються в ALU_RESIDUAL. Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює ZF; встановлює OF/CF, якщо старші розряди добутку (prod[31:16]) не дорівнюють 0.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

При множенні молодші 16 бітів добутку надходять у цільовий регістр dst, а старші 16 бітів фіксуються у внутрішньому службовому регістрі АЛП (ALU Residual). У наступній операції додавання цей залишок автоматично підсумовується (апаратна підтримка MAC / довгих чисел). Старшу частину можна також отримати в регістр через додавання очищеного регістра з самим собою. Команда NOP очищує залишковий регістр АЛП.

Поля з позначкою TBD визначають розширені параметри специфікації.

DIV

Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП

00100
Арифметика

Мнемоніка	DIV
Opcode	00100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

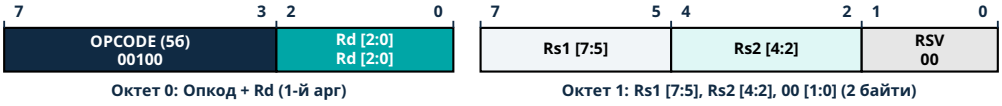
Призначення інструкції

Цілочисельне ділення зі збереженням залишку у внутрішньому регістрі АЛП.

Архітектурна семантика

```
Сигнатура:  DIV arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 / arg2
Операція:  arg0 <- arg1 / arg2
ALU_RESIDUAL <- arg1 % arg2
Приклад:  DIV R0, R1, R2 ; R0 = R1 / R2 (остача modulo в ALU_RESIDUAL)
```

Конфігурація та операнди



Октет 0: Опкод + Rd (1-й арг)
Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байти)

Октет 0: код операції 00100, біти $[2:0]$ задають цільовий регістр призначення Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають ділене Rs1 (R0–R7); біти $[4:2]$ задають дільник Rs2 (R0–R7); біти $[1:0]$ — резерв (00). Остача (modulo) автоматично фіксується в ALU_RESIDUAL. Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

При діленні на нуль зводиться прапорець помилки EM (Error Math) без блокування ядра; оновлює ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Результат цілочисельного ділення (частка) записується в цільовий регістр dst, а залишок від ділення (modulo) автоматично зберігається у внутрішньому службовому регістрі АЛП. Збережений залишок може бути задіяний у подальших операціях АЛП або скинутий командою NOP.

Поля з позначкою TBD визначають розширені параметри специфікації.

LMR

00101
Пам’ять

Завантаження даних із пам’яті та зовнішніх пристроїв у регістри

Мнемоніка	LMR
Opcode	00101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Пам’ять
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

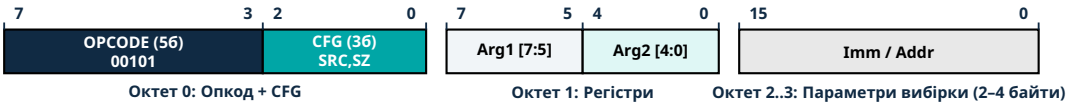
Призначення інструкції

Завантаження даних із пам’яті та зовнішніх пристроїв у регістри.

Архітектурна семантика

```
Сигнатура:  LMR arg0(REG), <SRC_MODE> :  arg0 = MEM[...]
Режими:  SRC=0:  LMR R0, [PC+] ; Imm Data (1-2 байти)
SRC=1:  LMR R0, [[PC+]] ; Imm Pointer
SRC=2:  LMR R0, [PTREG] ; Системний покажчик (AIP)
SRC=3:  LMR R0(R_hi), R1(R_lo) ; 24-біт адреса з пари
Операція:  arg0 <- MEM[EA] (старші біти адреси перезаписуються)
Приклад:  LMR R0, R1 ; EA=(R0«16)|R1, читання MEM[EA] в R0
```

Конфігурація та операнди



Октет 0: біт $[8]$ (SZ) задає розрядність ($0 = 1$ байт, $1 = 2$ байти / слово); бити $[10 : 9]$ (SRC) задають джерело даних (0 — дані безпосередньо за кодом команди, 1 — безпосередній покажчик за командою, 2 — непрямо через 24-бітний покажчик PTREG, 3 — непрямо через пару регістрів користувача).
Октет 1: біти $[7 : 5]$ задають регістр Arg1 (R_hi / приймач), біти $[4 : 2]$ задають регістр Arg2 (R_lo). У режимі SRC $= 3$ два 16-бітних регістри утворюють 32-бітне число, з якого для фізичної адресації використовуються 24 біти: Arg1 містить старші біти адреси, а Arg2 — молодші 16 біт. При цьому Arg1 одночасно виступає цільовим регістром: після вибірки з пам’яті старші біти адреси в ньому перезаписуються зчитаними даними.

Вплив на регістр прапорців (FREG)

Усі прапорці регістра стану FREG зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Інструкція LMR є фундаментальним шлюзом введення зовнішньої інформації в регістровий файл процесора LIM M2 ($R0$–$R7$). Архітектура підтримує чотири ортогональні режими вибірки та повну 24-бітну адресацію:
1. У режимі SRC $= 0$ процесор зчитує константу безпосередньо з потоку коду (1 або 2 байти) із просуванням лічильника команд PC.
2. У режимі SRC $= 1$ за інструкцією зчитується абсолютний покажчик, за яким завантажуються дані.
3. У режимі SRC $= 2$ читання виконується за 24-бітною фізичною адресою із системного регістра PTREG. За активного прапорця автоінкременту `FREG.AIP` (біт 12) регістр PTREG автоматично інкрементується на розмір вибірки ($+1$ або $+2$), забезпечуючи потокове DMA-подібне введення масивів за один такт.
4. У режимі SRC $= 3$ користувач вказує два 16-бітних регістри (`Arg1` та `Arg2`), які разом утворюють 32-бітне число, з якого на практиці адресації використовуються молодші 24 біти. Користувач визначає, що старші біти адреси знаходяться в `Arg1`, а молодші 16 біт адреси — в `Arg2`. У цьому режимі регістр `Arg1` зі старшими бітами адреси також стає регістром призначення, куди зберігаються зчитані дані (тобто старші біти адреси в `Arg1` перезаписуються даними, тоді як `Arg2` зберігає молодші 16 біт адреси).

Поля з позначкою TBD визначають розширені параметри специфікації.

LRM

Запис даних із регістрів у пам’ять та зовнішні пристрої

00110
Пам’ять

Мнемоніка	LRM
Opcode	00110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Пам’ять
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

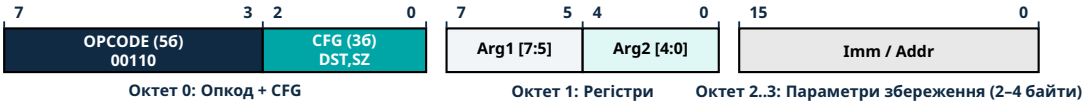
Призначення інструкції

Запис даних із регістрів у пам’ять та зовнішні пристрої.

Архітектурна семантика

```
Сигнатура:  LRM arg0(REG), <DST_MODE> :  MEM[...]  = arg0
Режими:  DST=0:  LRM R0, [PC+] ; Вбудована пам’ять за кодом
DST=1:  LRM R0, [[PC+]] ; Imm Pointer
DST=2:  LRM R0, [PTREG] ; Системний покажчик (AIP)
DST=3:  LRM R0(R_hi), R1(R_lo) ; 24-біт адреса з пари
Операція:  MEM[EA] <- arg0[15:0] (або молодший байт при SZ=0)
Приклад:  LRM R0, R1 ; EA=(R0«16)|R1, запис R0 в MEM[EA]
```

Конфігурація та операнди



Октет 0: біт $[8]$ (SZ) задає розрядність ($0 = 1$ байт, $1 = 2$ байти / слово); бити $[10 : 9]$ (DST) задають цільовий приймач даних (0 — запис безпосередньо за кодом команди, 1 — запис за покажчиком за командою, 2 — непрямо через 24-бітний покажчик PTREG, 3 — непрямо через пару регістрів користувача).
Октет 1: біти $[7 : 5]$ задають регістр-джерело Arg1 (Rs / R_hi), біти $[4 : 2]$ задають регістр Arg2 (R_lo). У режимі DST $= 3$ два регістри формують 32-бітне число, з якого використовуються 24 біти адреси (Arg1 — старші біти адреси та джерело даних, Arg2 — молодші 16 біт). Знищення наявних даних за адресою запису є відповідальністю програміста.

Вплив на регістр прапорців (FREG)

Усі прапорці регістра стану FREG зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Інструкція LRM є фундаментальним симетричним механізмом вивантаження регістрових даних процесора LIM M2 ($R0$–$R7$) в оперативну пам'ять та зовнішні порти периферії. Архітектура підтримує чотири ортогональні режими адресації:
1. У режимі DST $= 0$ процесор записує 1 або 2 байти безпосередньо в пам'ять слідом за інструкцією (PC) зі зсувом лічильника команд. Режим застосовний для динамічних буферів та самомодифікації коду; перезапис даних лежить на відповідальності програміста.
2. У режимі DST $= 1$ за інструкцією зчитується 16-бітний абсолютний адресний покажчик, за яким зберігаються дані.
3. У режимі DST $= 2$ запис виконується за 24-бітною системною адресою регістра PTREG. За активного прапорця FREG.AIP (біт 12) регістр PTREG автоматично інкрементується на $+1$ або $+2$, забезпечуючи апаратне потокове DMA-виведення масивів у пам'ять без програмних циклів.
4. У режимі DST $= 3$ використовується пара 16-бітних регістрів: Arg1 та Arg2 утворюють 32-бітне складене число, з якого молодші 24 біти задають фізичну адресу (Addr$[23 : 0]$). Дані з Arg1 записуються за обчисленою адресою, забезпечуючи прямий доступ до всього адресного простору 16 МБ.

Поля з позначкою TBD визначають розширені параметри специфікації.

LRR

Міжрегістрове переміщення (копіювання з точки Б в точку А)

00111
Пересилання

Мнемоніка	LRR
Opcode	00111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Пересилання
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

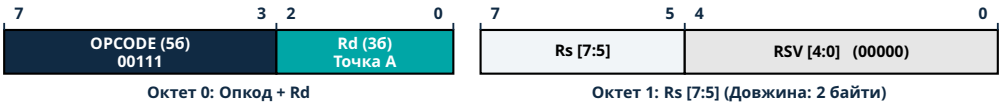
Призначення інструкції

Міжрегістрове переміщення (копіювання з точки Б в точку А).

Архітектурна семантика

```
Сигнатура:  LRR arg0(REG), arg1(REG) : arg0 = arg1
Операція:  arg0 <- arg1 (пряме копіювання з точки Б в точку А)
Приклад:  LRR R0, R1 ; R0 = R1 (значення R1 копіюється в R0)
```

Конфігурація та операнди



Октет 0: код операції 00111_2 (0x07), біти $[2:0]$ задають цільовий регістр призначення Rd (R0–R7, точка А, перший аргумент).
Октет 1: біти $[7:5]$ задають регістр-джерело Rs (R0–R7, точка Б); біти $[4:0]$ містять 00000_2 (резерв). При виконанні значення Rs копіюється в Rd за 1 такт. Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).

3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Інструкція LRR є фундаментальною операцією пересилання даних між регістрами загального призначення процесора ($R0$–$R7$). Вона здійснює передачу з джерела (точка Б) у приймач (точка А). Пересилання виконується через прямий апаратний обхідний тракт **MUX 1** $\rightarrow$ **MUX 2** в обхід АЛП, що виключає блокування обчислювального ядра та не змінює стан прапорців FREG.

Поля з позначкою TBD визначають розширені параметри специфікації.

CMP

Апаратне порівняння операндів

01000
Порівняння

Мнемоніка	CMP
Opcode	01000 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Порівняння
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

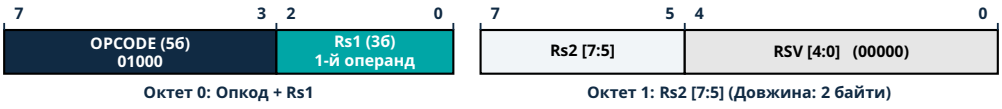
Призначення інструкції

Апаратне порівняння операндів.

Архітектурна семантика

```
Сигнатура:  CMP arg0(REG), arg1(REG) : compare(arg0, arg1)
Операція:  if arg0 < arg1 then FREG.CMP <- 01b (менше)
if arg0 == arg1 then FREG.CMP <- 10b (дорівнює)
if arg0 > arg1 then FREG.CMP <- 11b (більше)
(стан 00b — початкове скидання / не порівнювалось / помилка)
Приклад:  CMP R0, R1 ; 01:  R0<R1, 10:  R0==R1, 11:  R0>R1 (00:
скидання/помилка)
```

Конфігурація та операнди



Октет 0: код операції 01000, біти $[2:0]$ задають перший операнд Rs1 (перший аргумент).
Октет 1: біти $[7:5]$ задають другий операнд Rs2; біти $[4:0]$ містять 00000 (резерв).
Результат апаратного порівняння фіксується у полі FREG.CMP (біти $[5:4]$): 01 (Rs1 $<$ Rs2), 10 (Rs1 $=$ Rs2), 11 (Rs1 $>$ Rs2), 00 (скидання / не порівнювалось / помилка). Значення регістрів не модифікуються. Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює 2-бітне поле FREG.CMP (биты [5:4]), а також ZF, CF, OF без зміни регістрів.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Забезпечує швидке розгалуження алгоритмів.

Поля з позначкою TBD визначають розширені параметри специфікації.

JMP

Безумовний перехід

01001
Перехід

Мнемоніка	JMP
Opcode	01001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Безумовний перехід.

Архітектурна семантика

```
Сигнатура:  JMP target(IMM|REG) : PC = target
Операція:  PC <- target
Приклад:  JMP R0 ; Безумовний перехід за адресою в R0
```

Конфігурація та операнди



Октет 0: код операції 01001, біти $[2:0]$ (MOD): 000 — безпосередня 16-бітна адреса (довжина 3 байти); 001 — непрямий регістровий перехід (довжина 2 байти, октет 1 біти $[7:5]$ задають Rs); 010 — 24-бітний перехід за покажчиком PTREG.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Атомарне перезавантаження Program Counter за мінімум тактів.

Поля з позначкою TBD визначають розширені параметри специфікації.

JCR

Умовний перехід за прапорцем переносу

01010
Перехід

Мнемоніка	JCR
Opcode	01010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Умовний перехід за прапорцем переносу.

Архітектурна семантика

```
Сигнатура:  JCR target(IMM|REG) : if FREG.CF == 1 then PC = target
Операція:  if FREG.CF == 1 then PC <- target
Приклад:  JCR R2 ; Перехід за адресою R2 при наявності переносу
```

Конфігурація та операнди



Октет 0: код операції 01010, біти $[2:0]$ (MOD): 000 (16-біт адреса Imm, 3 байти), 001 (непрямо за Rs, 2 байти), 010 (24-біт за PTREG). Перехід виконується суворо при встановленому прапорці переносу FREG.CF = 1.

Вплив на регістр прапорців (FREG)

Прапорці читаються без модифікації.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Фундаментальний примітив для реалізації довгої арифметики.

Поля з позначкою TBD визначають розширені параметри специфікації.

JNZ

Умовний перехід, якщо не нуль

01011
Перехід

Мнемоніка	JNZ
Opcode	01011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Умовний перехід, якщо не нуль.

Архітектурна семантика

```
Сигнатура:  JNZ target(IMM|REG) : if FREG.ZF == 0 then PC = target
Операція:  if FREG.ZF == 0 then PC <- target
Приклад:  JNZ R3 ; Перехід за адресою R3 при ZF == 0 (не нуль)
```

Конфігурація та операнди



Октет 0: код операції 01011, біти $[2:0]$ (MOD): 000 (16-біт адреса Imm, 3 байти), 001 (непрямо за Rs, 2 байти), 010 (24-біт за PTREG). Перехід виконується при скинутому прапорці нуля FREG.ZF = 0.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Мінімізований час ухвалення рішення в мікрокоді.

Поля з позначкою TBD визначають розширені параметри специфікації.

JZ

Умовний перехід за нулем

01100
Перехід

Мнемоніка	JZ
Opcode	01100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Перехід
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Умовний перехід за нулем.

Архітектурна семантика

```
Сигнатура:  JZ target(IMM|REG) : if FREG.ZF == 1 then PC = target
Операція:  if FREG.ZF == 1 then PC <- target
Приклад:  JZ R4 ; Перехід за адресою R4 при ZF == 1 (нуль)
```

Конфігурація та операнди



Октет 0: код операції 01100, біти $[2:0]$ (MOD): 000 (16-біт адреса Imm, 3 байти), 001 (непрямо за Rs, 2 байти), 010 (24-біт за PTREG). Перехід виконується при встановленому прапорці нуля FREG.ZF = 1.

Вплив на регістр прапорців (FREG)

Прапорці зберігаються без змін.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Оптимізовано для алгоритмів пошуку.

Поля з позначкою TBD визначають розширені параметри специфікації.

AND

Побітове логічне I

01101
Логіка

Мнемоніка	AND
Opcode	01101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітове логічне I.

Архітектурна семантика

```
Сигнатура:  AND arg0(REG), arg1(REG) : arg0 = arg0 & arg1
Операція:  arg0 <- arg0 AND arg1
Приклад:  AND R0, R1 ; R0 = R0 & R1
```

Конфігурація та операнди



Октет 0: код операції 01101, біти $[2:0]$ задають цільовий приймач Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають другий операнд Rs (R0–R7); біти $[4:0]$ містять 00000_2 (резерв). Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює прапорець нуля ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.

4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Базова операція для накладання бітових масок.

Поля з позначкою TBD визначають розширені параметри специфікації.

OR

Побітове логічне АБО

01110
Логіка

Мнемоніка	OR
Opcode	01110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

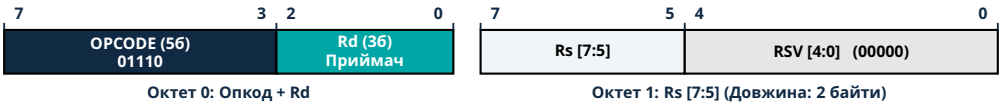
Призначення інструкції

Побітове логічне АБО.

Архітектурна семантика

```
Сигнатура:  OR arg0(REG), arg1(REG) : arg0 = arg0 | arg1
Операція:  arg0 <- arg0 OR arg1
Приклад:  OR R0, R1 ; R0 = R0 | R1
```

Конфігурація та операнди



Октет 0: код операції 01110, біти $[2:0]$ задають цільовий регістр Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають другий операнд Rs (R0–R7); біти $[4:0]$ містять 00000_2 (резерв).
Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Швидке об’єднання бітів керування.

Поля з позначкою TBD визначають розширені параметри специфікації.

NAND

Побітове логічне I-HI

01111
Логіка

Мнемоніка	NAND
Opcode	01111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітове логічне I-HI.

Архітектурна семантика

```
Сигнатура:  NAND arg0(REG), arg1(REG) : arg0 = ~(arg0 & arg1)
Операція:  arg0 <- NOT (arg0 AND arg1)
Приклад:  NAND R0, R1 ; R0 = ~(R0 & R1)
```

Конфігурація та операнди



Октет 0: код операції 01111, біти $[2:0]$ задають цільовий регістр Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають операнд Rs (R0–R7); біти $[4:0]$ містять 00000_2 (резерв). Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Дозволяє синтезувати компактний двійковий код.

Поля з позначкою TBD визначають розширені параметри специфікації.

NOR

Побітове логічне АБО-НІ

10000
Логіка

Мнемоніка	NOR
Opcode	10000 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

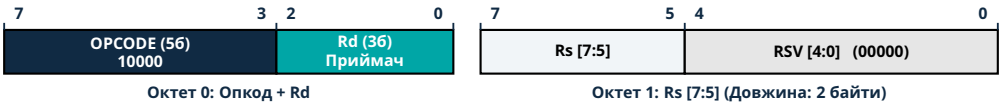
Призначення інструкції

Побітове логічне АБО-НІ.

Архітектурна семантика

```
Сигнатура:  NOR arg0(REG), arg1(REG) : arg0 = ~(arg0 | arg1)
Операція:  arg0 <- NOT (arg0 OR arg1)
Приклад:  NOR R0, R1 ; R0 = ~(R0 | R1)
```

Конфігурація та операнди



Октет 0: код операції 10000, біти $[2:0]$ задають цільовий регістр Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають операнд Rs (R0–R7); біти $[4:0]$ містять 00000_2 (резерв). Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Розширює набір швидких перетворень.

Поля з позначкою TBD визначають розширені параметри специфікації.

NOT

Побітова інверсія

10001
Логіка

Мнемоніка	NOT
Opcode	10001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітова інверсія.

Архітектурна семантика

```
Сигнатура:  NOT arg0(REG) : arg0 = ~arg0
Операція:  arg0 <- NOT arg0
Приклад:  NOT R0 ; R0 = ~R0 (1-байтова інструкція)
```

Конфігурація та операнди



Октет 0 (1 байт, 0 дод. байт)

Однобайтова інструкція (довжина: 1 байт). Октет 0: біти $[7:3]$ містять опкод 10001, біти $[2:0]$ прямо задають цільовий регістр Rd (R0–R7). Другий байт із шини пам'яті не запитується.

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Виконується за мінімальну кількість мікротактів без вибірки другого байта.

Поля з позначкою TBD визначають розширені параметри специфікації.

XOR

Побітове виключне АБО

10010
Логіка

Мнемоніка	XOR
Opcode	10010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

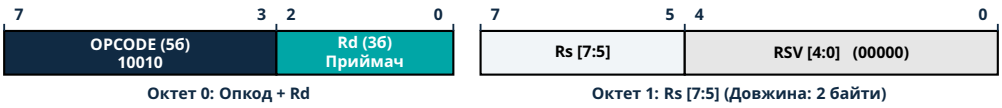
Призначення інструкції

Побітове виключне АБО.

Архітектурна семантика

```
Сигнатура:  XOR arg0(REG), arg1(REG) : arg0 = arg0 ^ arg1
Операція:  arg0 <- arg0 XOR arg1
Приклад:  XOR R0, R1 ; R0 = R0 ^ R1 (при R0==R1 обнуляє R0)
```

Конфігурація та операнди



Октет 0: код операції 10010, біти $[2:0]$ задають цільовий регістр Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають другий операнд Rs (R0–R7); біти $[4:0]$ містять 00000_2 (резерв).
Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Основа для алгоритмів контрольних сум та шифрування.

Поля з позначкою TBD визначають розширені параметри специфікації.

XNOR

Побітова еквівалентентність

10011
Логіка

Мнемоніка	XNOR
Opcode	10011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Логіка
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Побітова еквівалентентність.

Архітектурна семантика

```
Сигнатура:  XNOR arg0(REG), arg1(REG) : arg0 = ~(arg0 ^ arg1)
Операція:  arg0 <- NOT (arg0 XOR arg1)
Приклад:  XNOR R0, R1 ; R0 = ~(R0 ^ R1)
```

Конфігурація та операнди



Октет 0: код операції 10011, біти $[2:0]$ задають цільовий регістр Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають операнд Rs (R0–R7); біти $[4:0]$ містять 00000_2 (резерв). Довжина: 2 байти.

Вплив на регістр прапорців (FREG)

Оновлює прапорець ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Швидка перевірка відповідності бітових масок.

Поля з позначкою TBD визначають розширені параметри специфікації.

INC

Інкремент на одиницю

10100
Арифметика

Мнемоніка	INC
Opcode	10100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Інкремент на одиницю.

Архітектурна семантика

```
Сигнатура:  INC arg0(REG) : arg0 = arg0 + 1
Операція:  arg0 <- arg0 + 1
Приклад:  INC R0 ; R0 = R0 + 1 (1-байтова інструкція)
```

Конфігурація та операнди



Октет 0 (1 байт, 0 дод. байт)

Однобайтова інструкція (довжина: 1 байт). Октет 0: біти $[7:3]$ містять опкод 10100, біти $[2:0]$ прямо задають регістр Rd (R0–R7). Другий байт із шини пам'яті не запитується.

Вплив на регістр прапорців (FREG)

Оновлює ZF. Переповнення контролюється АЛП.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Швидкий крок лічильника без зайвої вибірки з шини.

Поля з позначкою TBD визначають розширені параметри специфікації.

DEC

Декремент на одиницю

10101
Арифметика

Мнемоніка	DEC
Opcode	10101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Декремент на одиницю.

Архітектурна семантика

```
Сигнатура:  DEC arg0(REG) : arg0 = arg0 - 1
Операція:  arg0 <- arg0 - 1
Приклад:  DEC R0 ; R0 = R0 - 1 (1-байтова інструкція)
```

Конфігурація та операнди



Октет 0 (1 байт, 0 дод. байт)

Однобайтова інструкція (довжина: 1 байт). Октет 0: біти $[7:3]$ містять опкод 10101, біти $[2:0]$ прямо задають регістр Rd (R0–R7). Другий байт із шини пам'яті не запитується.

Вплив на регістр прапорців (FREG)

Оновлює ZF. Позика контролюється АЛП.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам'ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Компактні декрементні цикли без зчитування другого байта.

Поля з позначкою TBD визначають розширені параметри специфікації.

PUSH

Збереження слова в стек

10110
Стек

Мнемоніка	PUSH
Opcode	10110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Стек
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

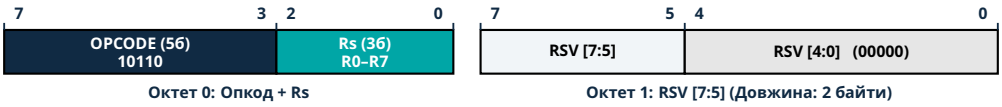
Призначення інструкції

Збереження слова в стек.

Архітектурна семантика

```
Сигнатура:  PUSH arg0(REG) : SREG -= 2, MEM[SREG] = arg0
Операція:  SREG <- SREG - 2, MEM[SREG] <- arg0
Приклад:  PUSH R0 ; Зберегти слово R0 на вершину стека
```

Конфігурація та операнди



Октет 0: код операції 10110, біти $[2:0]$ прямо задають збережуваний регістр-джерело Rs (R0–R7, перший аргумент).
Октет 1: біти $[7:0]$ містять 00000000_2 (резерв). Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Арифметичні прапорці не змінюються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Апаратна підтримка підпрограм та збереження контексту.

Поля з позначкою TBD визначають розширені параметри специфікації.

POP

Вилучення слова з вершини стека

10111
Стек

Мнемоніка	POP
Opcode	10111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Стек
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

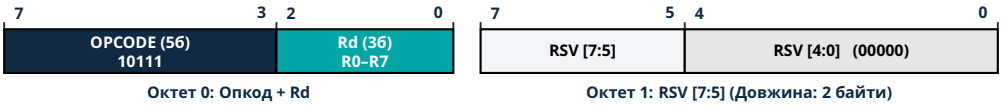
Призначення інструкції

Вилучення слова з вершини стека.

Архітектурна семантика

```
Сигнатура:  POP arg0(REG) : arg0 = MEM[SREG], SREG += 2
Операція:  arg0 <- MEM[SREG], SREG <- SREG + 2
Приклад:  POP R0 ; Витягнути слово з вершини стека в R0
```

Конфігурація та операнди



Октет 0: код операції 10111, біти $[2:0]$ прямо задають цільовий регістр-приймач Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:0]$ містять 00000000_2 (резерв). Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Арифметичні прапорці не змінюються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Симетричне відновлення контексту переривань.

Поля з позначкою TBD визначають розширені параметри специфікації.

BSL

Апаратний зсув ліворуч (ітеративний регістр)

11000
Зсуви

Мнемоніка	BSL
Opcode	11000 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Зсуви
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратний зсув ліворуч (ітеративний регістр).

Архітектурна семантика

```
Сигнатура:  BSL arg0(REG), arg1(IMM) : arg0 = arg0 « arg1
Кодування:  Октет 0:  [7:3]=11000, [2:0]=Rd (цільовий регістр R0–R7)
            Октет 1:  [7:0]=shift (кількість бітів зсуву 0–16)
Операція:  Послідовний тактовий зсув регістра Rd ліворуч на shift бітів:
Апаратний цикл (1 такт на 1 біт, ≈ 300 транзисторів):
Для кожного такту t ∈ [1..shift]:
Старший біт 15 витісняється (last_displaced <- bit 15);
При SMOD = 1 у молодший біт 0 влітає витіснений біт (ротація);
При SMOD = 0 у молодший біт 0 влітає біт із FREG.DBIT (0 або 1);
Фіксація:  FREG.DBIT <- last_displaced (останній виштовхнутий біт).
Приклад:  BSL R0, 4 ; Зсув R0 ліворуч на 4 біти (виконується за 4 такти
зсуву)
```

Конфігурація та операнди



Октет 0: код операції 11000, біти $[2:0]$ прямо задають зсуваний регістр Rd (R0–R7).
Октет 1: весь байт ([7:0]) задає кількість тактів зсуву shift (0–16). Фізично зсув реалізується потактовим циклом на 16 D-тригерах. Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Останній витіснений біт зберігається в FREG.DBIT (біт 13). Прапорець нуля FREG.ZF встановлюється при нульовому результаті. Керовано прапорцями FREG.SMOD (біт 14) та FREG.DBIT (біт 13).

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Ітеративний зсувний регістр на D-тригерах: вартість усього ≈ 300 транзисторів замість $\approx 30\,000$ для матричного однотактового зсувача. Зсув на 16 бітів потребує на 15 тактів більше, ніж зсув на 1 біт.

Поля з позначкою TBD визначають розширені параметри специфікації.

BSR

Апаратний зсув праворуч (ітеративний регістр)

11001
Зсуви

Мнемоніка	BSR
Opcode	11001 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Зсуви
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

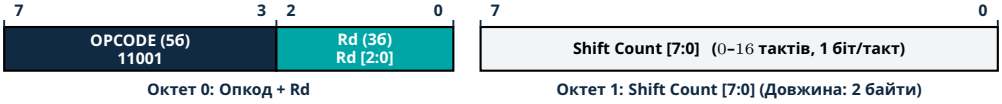
Призначення інструкції

Апаратний зсув праворуч (ітеративний регістр).

Архітектурна семантика

```
Сигнатура:  BSR arg0(REG), arg1(IMM) : arg0 = arg0 » arg1
Кодування:  Октет 0:  [7:3]=11001, [2:0]=Rd (цільовий регістр R0–R7)
            Октет 1:  [7:0]=shift (кількість бітів зсуву 0–16)
Операція:  Послідовний тактовий зсув регістра Rd праворуч на shift бітів:
Апаратний цикл (1 такт на 1 біт, ≈ 300 транзисторів):
Для кожного такту t ∈ [1..shift]:
Молодший біт 0 витісняється (last_displaced <- bit 0);
При SMOD = 1 у старший біт 15 влітає витіснений біт (ротація);
При SMOD = 0 у старший біт 15 влітає біт із FREG.DBIT (0 або 1);
Фіксація:  FREG.DBIT <- last_displaced (останній виштовхнутий біт).
Приклад:  BSR R0, 4 ; Зсув R0 праворуч на 4 біти (виконується за 4 такти
зсуву)
```

Конфігурація та операнди



Октет 0: код операції 11001, біти $[2:0]$ прямо задають зсуваний регістр Rd (R0–R7).
Октет 1: весь байт ([7:0]) задає кількість тактів зсуву shift (0–16). Фізично зсув реалізується потактовим циклом на 16 D-тригерах. Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Останній витіснений біт зберігається в FREG.DBIT (біт 13). Прапорець нуля FREG.ZF встановлюється при нульовому результаті. Керовано прапорцями FREG.SMOD (біт 14) та FREG.DBIT (біт 13).

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Ітеративний зсувний регістр на D-тригерах: вартість усього ≈ 300 транзисторів замість $\approx 30\,000$ для матричного однотактового зсувача. Зсув на 16 бітів потребує на 15 тактів більше, ніж зсув на 1 біт.

Поля з позначкою TBD визначають розширені параметри специфікації.

MSP

Маніпуляція спецрегістрами (FREG, PTREG, SREG, VEC)

11010
Система

Мнемоніка	MSP
Opcode	11010 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Система
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

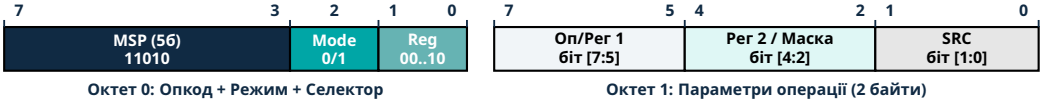
Призначення інструкції

Маніпуляція спецрегістрами (FREG, PTREG, SREG, VEC).

Архітектурна семантика

```
Сигнатура:  MSP sreg, [op], args :  робота зі спец-регістрами (2-6 байтів)
Операція:  Пряме завантаження (біт 10 = 0):
sreg=00 (FREG): FREG <- Rs[15:0] або imm16 / [addr16] / [$ptreg]
sreg=01 (PTREG): 32-бітний покажчик PTREG (4 ГБ): із РЗП, imm32, [$ptreg],
[addr32]
sreg=10 (SREG): 32-бітний покажчик стека SREG: із РЗП, imm32, [$ptreg],
[addr32]
sreg=11 (VEC): 32-бітний вектор переривання ($ir0..$ir15) або зони
($pv0..$pv15)
Модифікація (біт 10 = 1):
sreg=00 (FREG): FREG <- FREG ⊙ Mask {AND, OR, XOR, XNOR}
sreg=01 (PTREG): PTREG <- PTREG ± Offset {ADD, SUB}
sreg=10 (SREG): SREG <- SREG ± Offset {ADD, SUB}
Приклад:  MSP.ld $ir0, 0x00010000 ; Встановлення вектора IRQ0 (6 байтів)
MSP.ld $pv0, 0x00008000 ; Початок захищеної Зони Привілеїв 0
MSP.ld $pv1, 0x00008FFF ; Кінець захищеної Зони Привілеїв 0
MSP.ld $ptreg, [$ptreg] ; Нативне 1-тактове розіменування (2 байти)
MSP.or $freg, 0x0200 ; Увімкнення глобальних переривань MIE
```

Конфігурація та операнди



Октет 0: код операції 11010. Біт $[10]$ (біт 2 октету 0) задає режим: 0 — пряме завантаження, 1 — модифікація. Біти $[9:8]$ (біти 1–0 октету 0) задають селектор: 00 = FREG, 01 = PTREG, 10 = SREG.

Октет 1: при завантаженні ($[10] = 0$) задає джерело: Rs для FREG або пару Rs_hi, Rs_lo для PTREG/SREG. При модифікації ($[10] = 1$) для FREG біти $[7:6]$ кодують операцію (AND, OR, XOR, XNOR), а біти $[5:3]$ задають регістр маски Rs; для PTREG та SREG біт $[7]$ задає операцію (0=ADD, 1=SUB), а біти $[6:4]$ — регістр зсуву Rs. Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Прапорці FREG модифікуються напряму або маскою. Арифметика PTREG та SREG формує прапорець переносу CF через АЛП.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Універсальний керівний примітив системного рівня.

Поля з позначкою TBD визначають розширені параметри специфікації.

MOD

Цілочисельний залишок від ділення (modulo)

11011
Арифметика

Мнемоніка	MOD
Opcode	11011 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Арифметика
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

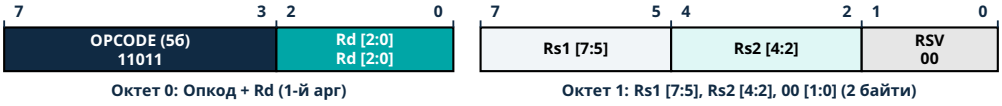
Призначення інструкції

Цілочисельний залишок від ділення (modulo).

Архітектурна семантика

```
Сигнатура:  MOD arg0, arg1, arg2 :  arg0 = arg1 % arg2
Операція:  dst <- arg1 % arg2
Приклад:  MOD R0, R1, R2 ; R0 = R1 % R2 (залишок від ділення)
```

Конфігурація та операнди



Октет 0: Опкод + Rd (1-й арг)

Октет 1: Rs1 [7:5], Rs2 [4:2], 00 [1:0] (2 байти)

Октет 0: код операції 11011, біти $[2:0]$ задають цільовий регістр Rd (R0–R7, перший аргумент).
Октет 1: біти $[7:5]$ задають ділене Rs1 (R0–R7); біти $[4:2]$ задають дільник Rs2 (R0–R7); біти $[1:0]$ — резерв (00). Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

При діленні на нуль зводиться апаратний прапорець помилки EM (Error Math) без зависання ядра; оновлює ZF.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.

4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Обчислює залишок від цілочисельного ділення діленого Rs1 на дільник Rs2 та записує результат безпосередньо в цільовий регістр Rd, не переписуючи проміжні результати в акумуляторах.

INTR

Програмне переривання за таблицею векторів (2 байти)

11100
Система

Мнемоніка	INTR
Opcode	11100 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Система
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Програмне переривання за таблицею векторів (2 байти).

Архітектурна семантика

```
Сигнатура:  INTR vec(IMM4) / INT vec :  виклик вектора переривання 0..15
Операція:  SREG <- SREG - 4; MEM32[SREG] <- PC
SREG <- SREG - 2; MEM16[SREG] <- FREG
FREG.INTR <- 1; FREG.PRV <- 1; FREG.MIE <- 0; PC <- VECTORS[vec]
Приклад:  INT 5 ; Програмне переривання за вектором №5 (2 байти)
```

Конфігурація та операнди



Октет 0: код операції 11100, молодші 3 біти $[2:0]$ містять 000. **Октет 1:** старші 4 біти $[7:4]$ зарезервовані (0000), молодші 4 біти $[3:0]$ задають номер вектора vec (0–15) із 16 векторів переривань ($ir0..$ir15). Довжина команди: 2 байти.

Вплив на регістр прапорців (FREG)

Встановлює прапорці FREG.INTR = 1 (біт 15), FREG.PRV = 1 (біт 11, режим супервізора) та скидає маску переривань FREG.MIE = 0 (біт 9); вихідні 32-бітний PC і 16-бітний FREG зберігаються у стек SREG.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Безпечний програмний виклик системних обробників (Syscall) та пасток відлагодження за номером вектора 0..15 із повним збереженням контексту на стеку SREG.

Поля з позначкою TBD визначають розширені параметри специфікації.

CALL

Виклик підпрограми зі збереженням адреси повернення

11101
Переходи

Мнемоніка	CALL
Opcode	11101 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Переходи
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Виклик підпрограми зі збереженням адреси повернення.

Архітектурна семантика

```
Сигнатура:  CALL target :  SREG -= 4, MEM32[SREG] = PC, PC = target
Кодування:  Октет 0:  [7:3]=11101, [2:0]=cfg (0:  imm16, 1:  Rd, 2:  [$ptreg],
3:  [addr16])
Операція:  ret_addr <- PC
SREG <- SREG - 4, MEM32[SREG] <- ret_addr
PC <- target
Приклад:  CALL func ; Виклик підпрограми за адресою func (3 байти)
CALL $0 ; Непрямий виклик через регістр R0 (2 байти)
CALL [$ptreg] ; Непрямий виклик через 32-бітний вказівник PTREG (2 байти)
```

Конфігурація та операнди



Октет 0: код операції 11101, біти $[2:0]$ задають режим адресації цілі переходу (cfg): 0 — пряма 16-бітна адреса (3 байти), 1 — непрямо через РОН Rd (2 байти), 2 — через 32-бітний вказівник PTREG (2 байти), 3 — непрямо за адресою в пам'яті (3 байти).
Перед переходом повна 32-бітна адреса повернення записується у вершину системного стека SREG.

Вплив на регістр прапорців (FREG)

Прапорці не змінюються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Стандартна інструкція виклику функцій та процедур зі збереженням 32-бітного контексту повернення у стеку SREG.

Поля з позначкою TBD визначають розширені параметри специфікації.

RET

Повернення з підпрограми з відновленням PC зі стека

11110
Переходи

Мнемоніка	RET
Opcode	11110 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Переходи
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Повернення з підпрограми з відновленням PC зі стека.

Архітектурна семантика

```
Сигнатура:  RET : PC = MEM32[SREG], SREG += 4
Операція:  PC <- MEM32[SREG], SREG <- SREG + 4
Приклад:  RET ; Повернення з підпрограми (1 байт)
```

Конфігурація та операнди



Октет 0 (1 байт, 0 дод. байт)

Однобайтова інструкція (довжина: 1 байт). Октет 0: код операції 11110, біти $[2:0]$ містять 000. Відновлює 32-бітний лічильник команд PC із вершини стека SREG і звільняє 4 байти стека (SREG += 4).

Вплив на регістр прапорців (FREG)

Прапорці не змінюються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

Особливості реалізації та примітки

Симетрична інструкція повернення з підпрограм, викликаних командою CALL.

Поля з позначкою TBD визначають розширені параметри специфікації.

HALT

Апаратний зупин тактування процесора

11111
Керування

Мнемоніка	HALT
Opcode	11111 (старші 5 бітів коду команди)
Формат команди	5 бітів opcode, 3 біти cfg та динамічні операнди (у спеціалізованих команд — операнди безпосередньо після opcode)
Клас операції	Керування
Статус фіксації	Базовий функціонал затверджено; бітова розкладка аргументів на стадії фіналізації

Призначення інструкції

Апаратний зупин тактування процесора.

Архітектурна семантика

```
Сигнатура:  HALT / HLT : зупин ядра
Операція:  CPU.HALTED <- 1, PINS.HALT <- 1
Приклад:  HALT ; Повний зупин процесора (1 байт)
```

Конфігурація та операнди



Октет 0 (1 байт, 0 дод. байт)

Однобайтова інструкція (довжина: 1 байт). Октет 0: код операції 11111, біти $[2:0]$ містять `000`. Переводить процесор у режим зупину до сигналу скидання RESET або переривання IRQ.

Вплив на регістр прапорців (FREG)

Прапорці не змінюються.

Мікрооперації виконавчого тракту

1. Комутація мультиплексорів тракту даних для вибірки операндів.
2. Виконання операції у функціональному блоці (АЛП / пам’ять / переходи).
3. Маршрутизація результату через арбітр запису (MUX 2) до цільового регістру.
4. Синхронне оновлення прапорців FREG та системних покажчиків (PTREG, SREG).

Точна сітка мікротактів та строби: **TBD**.

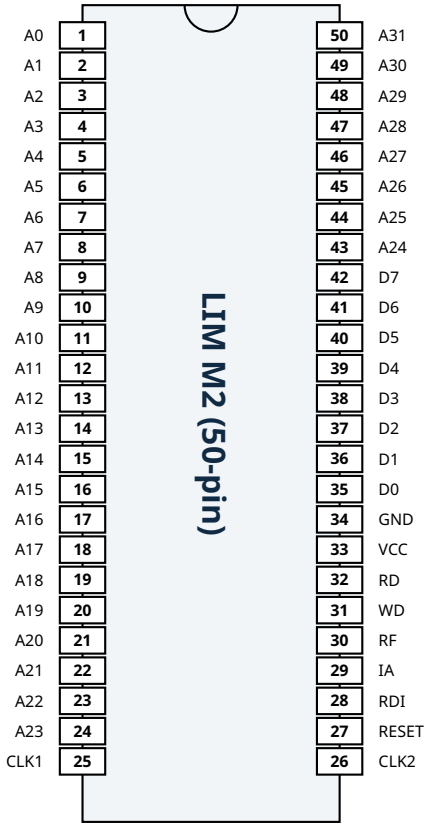
Особливості реалізації та примітки

Завершення роботи або перехід в енергоощадний стан.

Поля з позначкою TBD визначають розширені параметри специфікації.

7 Фізичний інтерфейс та цоколівка (50-вивідний корпус)

Процесор розміщується у 50-вивідному корпусі DIP-50 / QFP-52 із нативною 32-розрядною шиною адреси та 8-бітною (однобайтовою) зовнішньою шиною даних:



7.1 Опис системних сигналів

Вивід	Тип	Опис системного сигналу
A0–A31	Вихід / Tri-State	32-розрядна нативна магістраль адреси (прямий фізичний доступ до 4 ГБ пам’яті без вікон та банків).
D0–D7	Двоспрямований	8-розрядна (однобайтова) зовнішня шина обміну з пам’яттю та периферією.
CLK1, CLK2	Вхід	Двофазна тактова синхронізація без перекриття фаз.
RESET	Вхід	Апаратний сигнал скидання процесора (активний низький).
RDI	Вхід	Raw Direct Interrupt: пріоритетний вхід апаратного переривання.
IA	Вихід	Interruption Accepted / Interrupt Acknowledge: строб підтвердження прийому переривання пристрою.
RF	Вхід	Ready Flag: апаратне підтвердження готовності веденого пристрою пам’яті/периферії.
RD, WD	Вихід	Строби зчитування (Read) та запису (Write) системної шини.
VCC, GND	Живлення	Лінії цифрового живлення процесора (+3.3 В / +5 В номінально) та земля.

8 Підсистема переривань та векторна обробка

Підсистема переривань процесора LIM M2 спроєктована для забезпечення мінімальної латентності реакції на події реального часу, строгого детермінізму та повного захисту від шинних конфліктів. Архітектура об'єднує апаратні зовнішні запити та програмні виключення в єдину 16-векторну ієрархію з надійним апаратним квітуванням на фізичному рівні.

8.1 Єдиний простір 16 векторів переривань

Процесор підтримує рівно 16 системних векторів (номери 0–15), які є ортогональними та спільними як для зовнішньої периферії, так і для внутрішніх подій процесорного ядра:

Table 3: Єдина таблиця 16 векторів переривань LIM M2

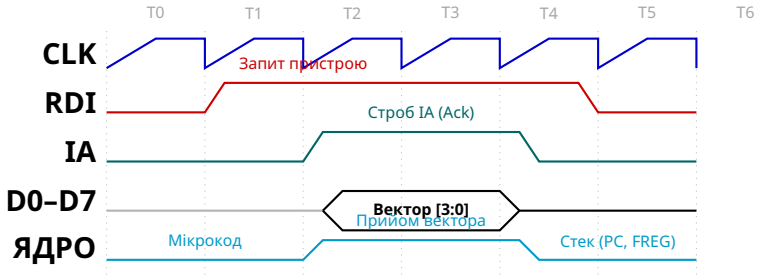
Вектор	Код	Джерело / Тип	Функціональне призначення
VEC 0	`0x00`	Апаратний / Скидання	Power-on Reset: початкова ініціалізація процесора та регістрів.
VEC 1	`0x01`	Апаратний (NMI)	Немасковане переривання: збій живлення, критична апаратна аварія.
VEC 2	`0x02`	Програмна пастка	Арифметичне виключення (ділення на нуль, прапорець `FREG.EM`).
VEC 3	`0x03`	Програмна пастка	Системний виклик ядра (System Call / Trap диспетчера ОС).
VEC 4	`0x04`	Апаратний / Периферія	Системний таймер інтервалів / Квантувач планувальника.
VEC 5	`0x05`	Апаратний / Периферія	Контролер послідовного інтерфейсу (UART RX/TX).
VEC 6	`0x06`	Апаратний / Периферія	Контролер прямого доступу до пам'яті (DMA Ready).
VEC 7	`0x07`	Апаратний / Периферія	Зовнішній контролер переривань / Шинний міст.
VEC 8–15	`0x08–0x0F`	Спільні (HW / SW)	Користувацькі апаратні лінії та програмні вектори.

8.2 Апаратний протокол квітування (RDI → IA → D0–D7)

Для надійної взаємодії із зовнішніми контролерами реалізовано 4-фазний цикл апаратного рукостискання (handshake), що повністю виключає необхідність постійного опитування регістрів (polling):

1. **Фаза запиту (RDI):** Периферійний пристрій встановлює активний високий рівень на вході `RDI` (*Raw Direct Interrupt*). Лінія опитується ядром на атомарних межах мікрокоманд.
2. **Фаза перевірки маски (Арбітраж):** Якщо глобальне маскування переривань дозволено (біт `FREG.MIE` = 1), процесор фіксує запит і завершує поточну неподільну мікрооперацію тракту даних.
3. **Фаза підтвердження (Строб IA):** Ядро генерує активний строб на виході `IA` (*Interruption Accepted / Interrupt Acknowledge*, вивід 26). Цей строб сповіщає всі зовнішні пристрої на шині про готовність ядра прийняти номер вектора.
4. **Фаза передачі вектора (Шина D0–D7):** Одночасно з видачею `IA` зовнішня шина даних `D0–D7` перемикається ядром у режим приймання (вхід), а лінія зчитування RD активується. Пристрій, що надіслав запит, утримує на лініях `D0–D7` 8-бітний ідентифікатор вектора (молодші 4 біти адресують вектори 0–15).

5. **Фаза фіксації та перехід:** Процесор фіксує дані з шини `D0–D7`, знімає сигнал `IA` та переходить до процедури збереження контексту.



8.3 Збереження контексту у 24-бітному стеку та вхід в обробник

Прийняття переривання дозволено виключно за умови, що процесор не перебуває в стані обробки іншого переривання (`FREG.INTR` = 0) і переривання глобально дозволені (`FREG.MIE` = 1). Якщо `FREG.INTR` = 1, будь-які повторні апаратні переривання апаратно заблоковані.

Під час фіксації переривання ядро виконує таку послідовність захисту стану:

1. **Збереження лічильника команд (PC):** Поточне значення PC зберігається на вершині системного стека за 24-бітним вказівником SREG:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{PC}_{15:0}$$

2. **Збереження прапорців та статусу (FREG):** Поточний стан регістра FREG (з початковим значенням `FREG.INTR`=0) розміщується у стеку:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{FREG}$$

3. **Апаратне блокування вкладених переривань:** Ядро виставляє прапорець активності переривання `FREG.INTR` ← 1 (біт 15) та скидає маску `FREG.MIE` ← `0`, унеможливлюючи каскадні повторні входи.
4. **Перехід за вектором:** Адреса обробника обчислюється на основі прийнятого номера вектора та завантажується в PC.

8.4 Повернення з переривання

Вихід з процедури обробки переривання здійснюється відновленням збереженого контексту зі стека:

1. Зі стека зчитується та відновлюється початковий стан регістра статусу:

$$\text{FREG} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

   (При цьому біт `FREG.INTR` автоматично повертається в `0`, знімаючи блокування, а біт `FREG.MIE` відновлює свій попередній стан).
2. Зі стека відновлюється лічильник команд:

$$\text{PC} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

3. Процесор продовжує виконання перерваного потоку інструкцій без жодних втрат даних.

9 Мова асемблера та компілятор LIM M2 (limasm.py)

Для розробки програмного забезпечення та системних прошивок під архітектуру **LIM M2** реалізовано двопрохідний компілятор асемблера LIM2/sim/limasm.py, який компілює вихідний код (.asm / .lasm) одночасно у бінарний образ пам'яті (.bin) та шістнадцятковий файл (.hex).

9.1 Іменування регістрів та суфікси розміру операндів

- **Регістри загального призначення (16 біт):** позначаються через префікс $: $0, $1, $2, $3, $4, $5, $6, $7 (також підтримується запис R0–R7).
- **Спеціальні регістри:** $ptreg (24-бітний регістр-вказівник PTREG), $freg (16-бітний регістр прапорців FREG), $sreg / $sp (24-бітний вказівник стека SREG), $vec0–$vec7 (вектори переривань).
- **Суфікси розміру (.b / .w / .d):** у разі потреби вказати розмір даних після мнемоніки пишеться XXX.b (byte, 8 біт), XXX.w (word, 16 біт, за замовчуванням) або XXX.d (doubleword, 24/32 біти).

9.2 Арифметичні, унарні інструкції та робота з пам'яттю (LMR, LRM)

- **Регістрова арифметика та унарні операції:**

```
ADD $0, $1, $2      ; $0 = $1 + $2 (або ADD R0, R1, R1)
SUB $3, $0, $1      ; $3 = $0 - $1 (аналогічно MUL, DIV, MOD)
INC $0              ; $0 = $0 + 1  (1-байтова інструкція, або INC R0)
DEC $1              ; $1 = $1 - 1  (1-байтова інструкція)
```

- **Завантаження та вивантаження пам'яті (LMR та LRM):**
  1. **Просто число (без дужок)** — безпосереднє значення (SRC $= 0$ / DST $= 0$): LMR.w $0, 0x1234, LMR.b $1, 0x42.
  2. **Квадратні дужки [addr]** — посилання на дані у пам'яті (SRC $= 1$ / DST $= 1$): LMR.w $0, [0x1000], LRM.w $0, [0x1000].
  3. **Посилання у [$ptreg]** — непряме звернення через 24-бітний PTREG (SRC $= 2$ / DST $= 2$): LMR.w $0, [$ptreg], LRM.b $4, [$ptreg].
  4. **Вказівник на основі пари регістрів [$0:$1]** (SRC $= 3$ / DST $= 3$): LMR $0, [$0:$1] (також LMR, $0, [$0:$1]), LRM $0, [$0:$1].

9.3 Спеціальні регістри (MSP) та багаторазове розіменування через PTREG

```
MSP.and $freg, 0x0001       ; Маска AND на прапорцевий регістр: FREG &= 0x0001
MSP.ld  $freg, 0x0000       ; Глухе налаштування у регістр прапорців
MSP.ld  $ptreg, [$0:$1]     ; Завантаження 32-бітного поінтера у PTREG з пари [$0:$
MSP.ld  $sreg, [$0:$1]      ; Завантаження 32-бітного вказівника стека SREG з пари
MSP.ld  $ptreg, 0xA000      ; Безпосереднє завантаження адреси 0xA000 у PTREG
MSP.ld  $ptreg, [0xA000]    ; Завантаження поінтера за посиланням з пам'яті [0xA000

; Нативне багаторазове розіменування ланцюжка вказівників через PTREG:
MSP.ld  $ptreg, 0xA000      ; Початкова адреса ланцюжка
MSP.ld  $ptreg, [$ptreg]    ; 1-ше розіменування: PTREG = *0xA000
MSP.ld  $ptreg, [$ptreg]    ; 2-ге розіменування: PTREG = **0xA000
MSP.ld  $ptreg, [$ptreg]    ; 3-тє розіменування: PTREG = ***0xA000
LMR.w   $0, [$ptreg]        ; Читання даних за фінальною розіменованою адресою
```

Компіляція у `.bin` та `.hex`: `python3 LIM2/sim/limasm.py LIM2/sim/demo.asm -l`.

10 Висновки

Архітектура **LIM M2** гармонійно поєднує перевірену надійність мікропрограмного керування та сучасну ефективність активного мультиплексорного тракту без третього стану. Завдяки усуненню шинних колізій та ємнісного розмивання фронтів процесор забезпечує виняткову цілісність сигналів і відкриває безпрецедентний частотний діапазон від 200 МГц до фізичних меж сучасного кремнію.